

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART XVII

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

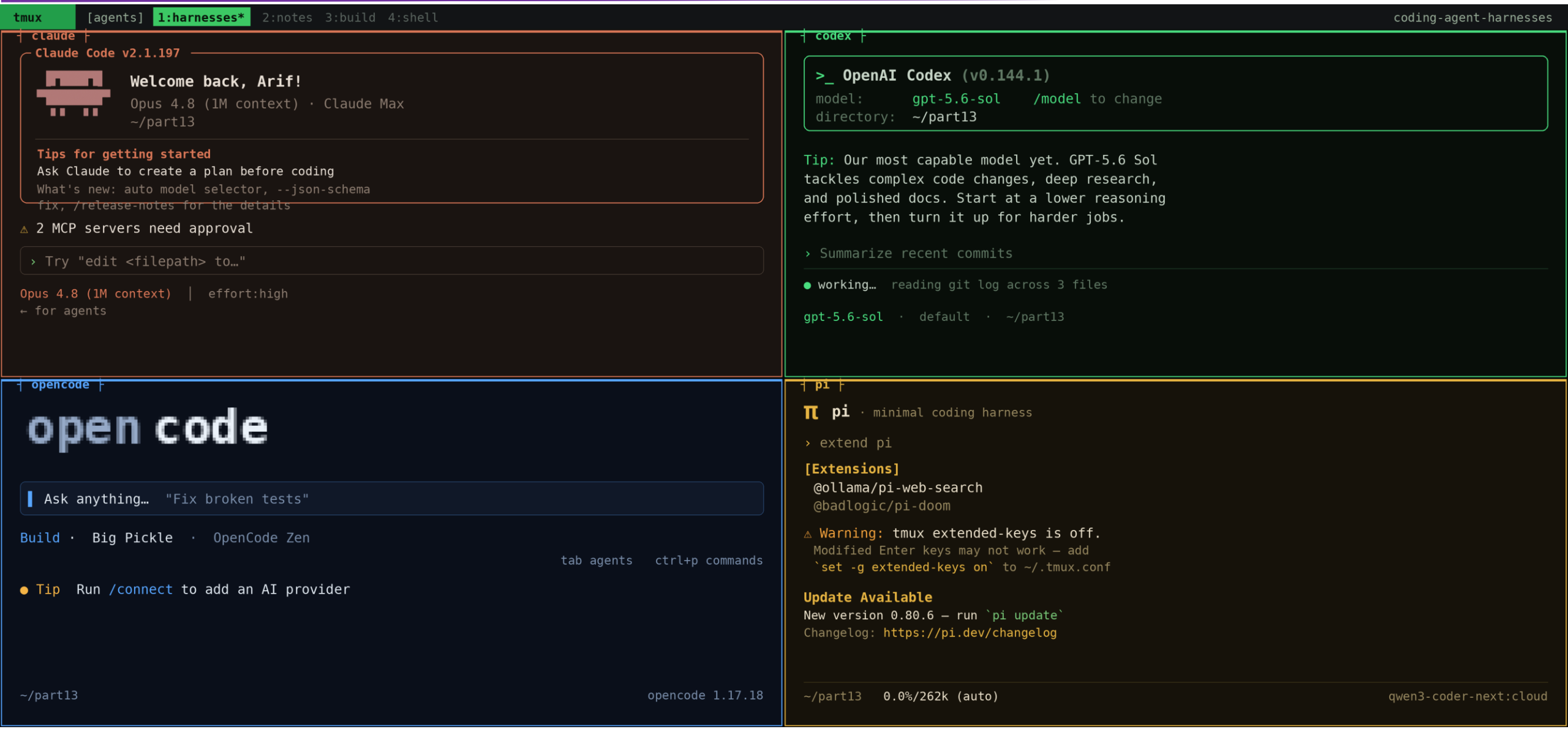
● **Today's session — Part XVII**

Session	Topic
Part I	Overview of Agentic Coding Assistants
Part II	Claude Ecosystem, Installing & Using Claude Code
Part III	Slash Commands, Context Window & .claude Directory
Part IV	Workflow of Building a Basic Vulnerable Web Application
Part V	Exploiting Vulnerable Web Application
Part VI	Workflow of Implementing Features and Fixing Bugs
Part VII	Collaborative Workflow: Claude Code + GitHub + Jira
Part VIII	Built-in Skills & Vuln-Web-App Current Status
Part IX	Creating & Using Custom Skills & Skill Marketplace

Session	Topic
Part X	Deep Dive: Conceptual Understanding of MCP
Part XI	MCP in Action
Part XII	Creating / Using Built-in & Custom Subagents
Part XIII	Git Worktrees, Agent Teams & a Revisit to OWASP Top 10
Part XIV	A2A Protocol with a Hands-on Demonstration
Part XV	Permissions in Claude Code
Part XVI	Sandboxing in Claude Code
Part XVII	**Hooks and Guardrails in Claude Code**

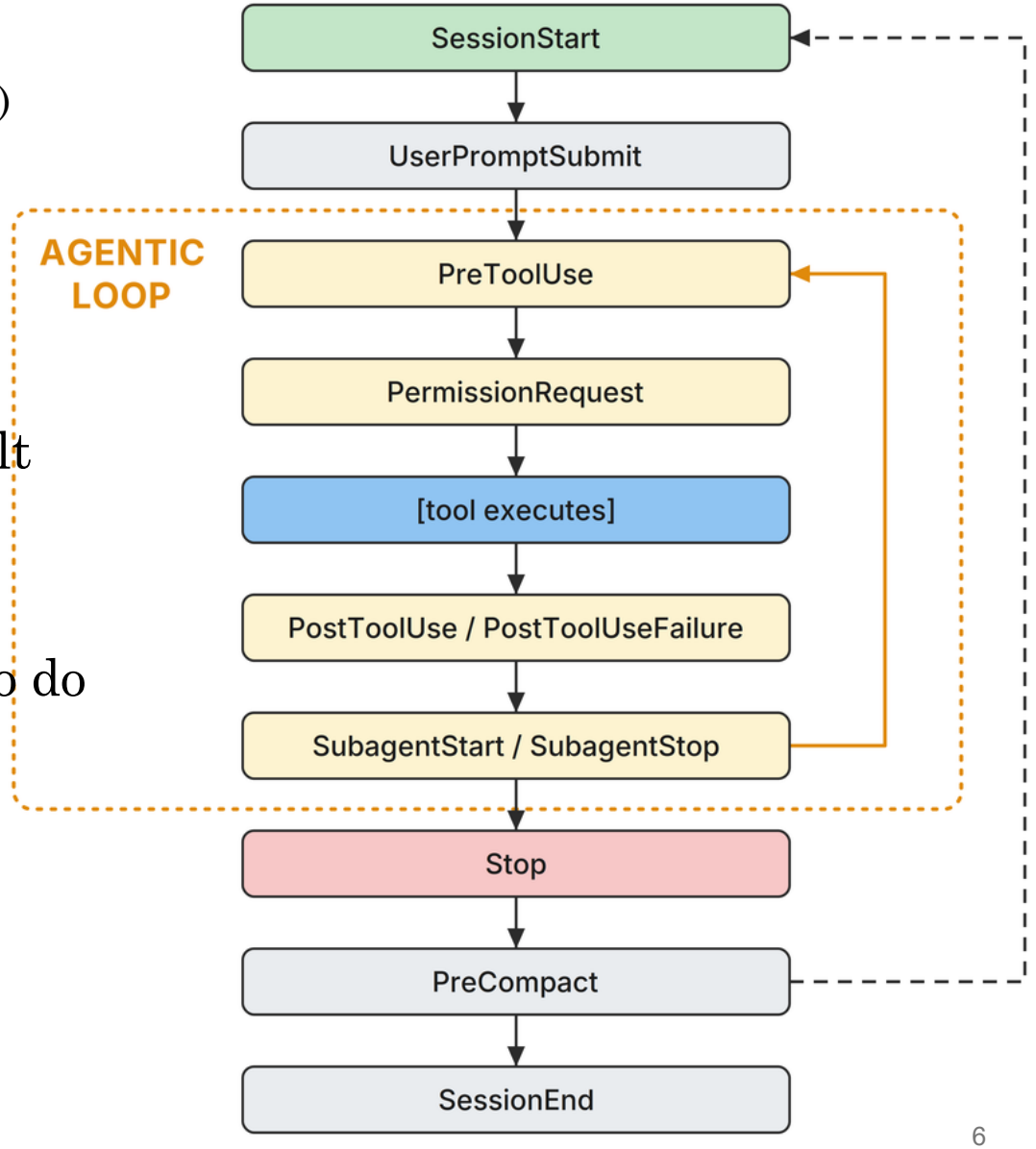
A Quick Recap

Agent Harnesses



Session Life Cycle & Agentic Loop

- **Session Start:** Working Memory is loaded
  - `CLAUDE.md` → semantic memory (facts, conventions, constraints)
  - `MEMORY.md` → episodic memory (past events & experiences)
  - `SKILL.md` → procedural memory (reusable workflows)
- **UserPromptSubmit:** Last moment the loop is yours
- **Agentic Loop:** intent → permission → execute → result
- **Delegation:** Subagents runs their own scoped tasks
- **The Loop Repeats:** Until the model has nothing left to do
- **Compaction:** Context compressed, session restarts
- **Notification:** Asynchronous outside the flow



Six Pillars of the Claude Code Ecosystem

Knowledge — What Claude knows

CLAUDE.md

ALWAYS LOADED

Persistent project context loaded at the start of every session. Files merge hierarchically — user, project, subdirectory — with more specific files winning.

Skills

USER OR CLAUDE INVOKED

Reusable instructions plus bundled scripts and assets, loaded only when needed via progressive disclosure. Run manually with /skill-name or let Claude trigger one from its description. Absorbed custom slash commands.

Execution — How work gets done

MCP servers

MODEL INVOKED

Brokered access to external systems — APIs, databases, and developer tools — exposed to Claude as callable tools over a standard protocol.

Subagents

DELEGATED

Specialized workers that run in their own isolated context window and return only a summary — heavy research and parallel tasks never pollute the main thread.

Operations — Enforcing and sharing

Hooks

EVENT TRIGGERED

Deterministic code that fires on lifecycle events — formatting, logging, validation, notifications. Enforce rules with code, not prompts.

Plugins

DISTRIBUTION UNIT

Bundles skills, subagents, hooks, MCP configs, and LSP servers* into one installable, versioned package — the canonical way to share a setup across teams and marketplaces.

* LSP servers give Claude IDE-grade code intelligence — go-to-definition, find references, live diagnostics — via the Language Server Protocol. Configured through plugins; the language server binary is installed separately.

When to Use Which in Claude Code?



WRITE A `CLAUDE.md / AGENTS.md + .claude/rules/`

Rule Use rules for stuff that is True on every prompt, e.g., describing your tech stack, your naming convention, build commands etc.

PACKAGE A `.claude/skills/<name>/`

Skill Repetitive workflows that AI loads on demand (progressive disclosure), e.g., code-review, deploys, brand-guidelines skills

RUN AN `.claude/agents/, agent-memory/<name>/MEMORY.md`

Agent Use agents to keep your session context from getting polluted. Run parallel background agents, git worktrees and agent teams.

CONNECT VIA `Local/remote MCP Servers`

MCP Used to access external systems, APIs, databases and developer tools.

USE A `.claude/settings.json`

Hook A script written by developer that the harness automatically executes on a specific event during a session's lifecycle like start of session, user prompt submission, pre-tool usage, compaction etc



Hooks

What is a Hook?

What Is a Hook? — Your Own Code, in Claude's Lifecycle

Claude Code stops at fixed points while it works. A hook is the script you put at one of those points.

1 · WHAT IT IS

A hook is a script you write.
Claude Code runs it for you.

You decide what it checks. Claude Code decides when to call it — and it never forgets to.

WHERE IT SITS

Claude is about to act → **your script runs first** → Claude gets your answer

Your script can let the action through, or stop it and say why.

2 · WHAT MAKES IT DIFFERENT

You never run it
Claude Code calls it, at a point in its own lifecycle that you chose.

It runs every time
Not when the model remembers to. Not when someone approves a prompt. Every time.

It is ordinary code
Bash, python, anything. So it can open a file, run a test, or query your repo before answering.

It can say no — with a reason
And Claude reads that reason, so it changes approach instead of trying the same thing again.

3 · WHY WE NEED ONE

WHAT THE PERMISSION RULE CHECKED

The **words of the command** — `python3 scripts/leak.py`

✓ **matched an allow rule, so it ran with no prompt**

↓

WHAT NOBODY CHECKED

The **file it was about to run** — which opens `.env` and prints your secrets.

🚫 **no rule could have seen that**

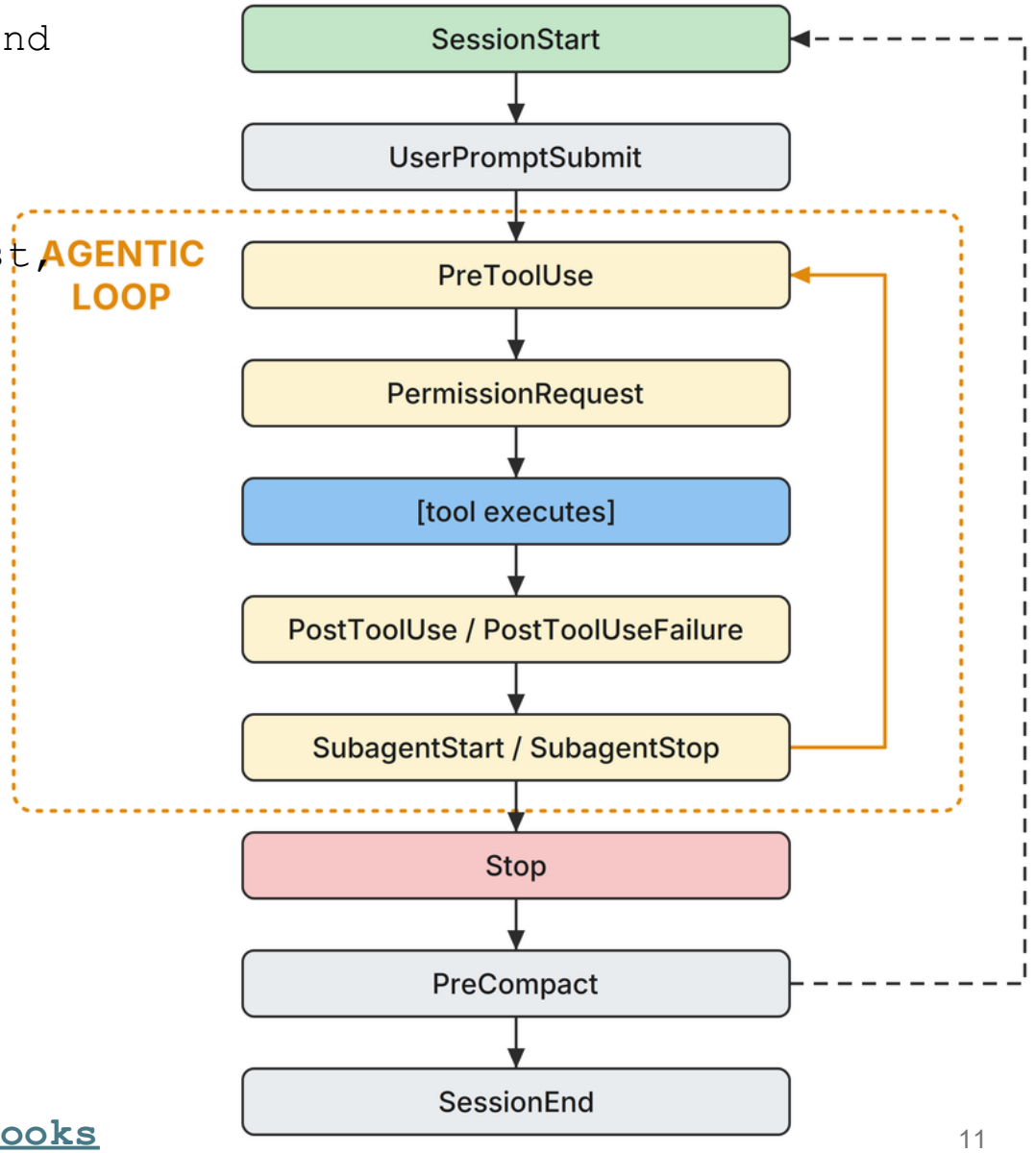
A rule matches text.
A hook runs code — so it can open the file first.

⚡ **A permission rule is a list you configure. A hook is a program you write.**
That is the whole difference. A rule can only judge what a command looks like; a hook can judge anything a program is able to look at.

Hooks Lifecycle & Use Cases

- **Once per Session:** `SessionStart, PreCompact, SessionEnd`
- **Once per Turn:** `UserPromptSubmit, Stop/StopFailure`
- **Agentic Loop:** `PreToolUse, PermissionRequest, PostToolUse, SubagentStart, SubagentStop`

Run **`/hooks`** command in Claude Code to check available hook events



Use Cases:

- Blocking dangerous shell commands (**`rm -rf /`**)
- Protecting sensitive files (**`.env`**)
- Triggering a security scan after AI finishes writing code
- Telemetry
- Auto-formatting (Black) & Linting
- Generating project summary at the start of every session

Notification (Async)

https://code.claude.com/docs/en/hooks

Anatomy of a Hook

Anatomy of a Hook — Event, Matcher, Handler

One rule: after Claude writes a file, run a script. Stated in plain English, broken into its three parts, and written out as JSON.

1 · WHAT WE WANT

Every time Claude writes a file, run my script.

Claude writes a file

↓

notify.sh runs

↓

you get a message

You never run notify.sh yourself. You register it once, and Claude Code runs it every time — whether or not it remembers to.

2 · THE THREE PARTS

EVENT when it fires

PostToolUse means just after a tool call has finished. Change this word and the hook fires somewhere else in the session.

MATCHER which tool

Write only. Leave the matcher out and the same hook fires for every tool Claude uses.

HANDLER what to run

type says how to run it — here, a shell command. command says what to run — here, ./notify.sh.

3 · WHAT YOU WRITE

```
// .claude/settings.json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write",
      "hooks": [{
        "type": "command",
        "command": "./notify.sh"
      }]
    }]
  }
}
```

Read it as one sentence: on PostToolUse, when the tool is Write, run this command.

A hook is three answers: when it fires, what it matches, and what it runs.

Every hook in the rest of this session is a variation on this one shape — a different event, a tighter matcher, or a handler that does more than print a message.

Handler Types

Handler Types — Selection Criteria and Examples

All five run your check. They differ only in where the check lives and how much judgment it needs — so ask in this order and stop at the first yes.

1. Can a shell script on this laptop decide it? — yes →

`command` a script you wrote — watch out: you maintain the script

Claude edits app.py, and black reformats that one file straight away.

```
PostToolUse · "Edit|Write"  →  { "type": "command", "command": "./format.sh" }
```

2. Does your whole team need the same rule? — yes →

`http` a POST to your service — watch out: if the service is down, the call goes through

Every Bash command goes to your team's policy service — one place to update, not twenty laptops.

```
PreToolUse · "Bash"  →  { "type": "http", "url": "https://policy.acme.dev/check" }
```

3. Is the check already a tool on an MCP server you connected? — yes →

`mcp_tool` a tool you already have — watch out: the server must already be connected

You wired up a scanner MCP in Part XI — call its tool instead of shelling out to a second copy.

```
PostToolUse · "Write"  →  { "type": "mcp_tool", "server": "scanner", "tool": "scan" }
```

4. Does deciding need judgment, not a pattern? — yes →

`prompt` one model call — watch out: seconds, and not always the same answer

A regex catches the word credentials. It misses “copy my keys somewhere safe”. A model does not.

```
UserPromptSubmit  →  { "type": "prompt", "prompt": "Does this move secrets off the box?" }
```

5. Does deciding mean going and reading the code first? — yes →

`agent` a subagent with tools — watch out: experimental, and the slowest of the five

Before Claude may finish the turn, a subagent reads the diff and looks for anything secret.

```
Stop  →  { "type": "agent", "prompt": "Read the diff. Does it add a secret?" }
```

The list is ordered by cost — so take the first yes, not the cleverest one.

A shell script answers instantly and answers the same way every time. A model call takes seconds and can change its mind. Never reach for judgment when a filename check would have done.

Where Hooks Live

Configuration and Precedence — Six Locations, One Merged List

Where you put a hook decides who it applies to. Two files can both have hooks, and both will run — and when nothing runs at all, three things are almost always the reason.

1 · WHERE YOU PUT IT

`~/.claude/settings.json`
every project on this machine · just you

`.claude/settings.json`
this project · committed, the team gets it

`.claude/settings.local.json`
this project · just you, gitignored

`managed policy settings`
the whole organisation · admin only

`plugin hooks/hooks.json`
wherever that plugin is enabled

`skill or agent frontmatter`
only while that component is active

2 · IF TWO FILES HOOK THE SAME EVENT

`~/.claude/settings.json`
`PostToolUse · Write → run the formatter`

+

`.claude/settings.json`
`PostToolUse · Write → scan for secrets`

↓

Both of them run.
Same event, same matcher, two files — the two lists are joined. Neither file wins.

`/hooks`
lists every hook that is live right now, and which file each one came from

3 · HOW YOUR SCRIPT ANSWERS

`exit 0` **carry on**
The normal ending. Claude Code runs the tool as if the hook had never been there. If you want to say more than yes, print JSON — but it must be the only thing on stdout.

`exit 2` **stop it**
The one that blocks. The tool call never happens, and whatever your script wrote to stderr is handed to Claude as the reason why.

`exit 1` **your script broke**
Not a block. Claude Code notes the error, throws the output away, and runs the tool anyway. This is the mistake everyone makes on their first hook.

Put it where the people it protects are.
Your own habits go in the user file. A rule the team must not lose goes in the project file and gets committed. And whichever file it came from, a hook's deny holds in every permission mode.

Hands on Practice



https://github.com/arifpucit/cc-perms-sandbox

Where Hooks Fail?

Four Documented Failure Modes — What You Wrote, and What Happens

All four are documented, and all four are quiet — nothing tells you it happened. Read the left box, then the right one.

1 Claude just uses another command

Your rule blocks one way of doing the job. There is always a second way.

YOU BLOCK	→	SO CLAUDE DOES THIS INSTEAD
`Write`	→	`cat > file.py <<EOF`

2 One flag turns every hook off

Bare mode never reads your hooks, skills, plugins, MCP servers or CLAUDE.md at all.

YOUR HOOKS LOAD HERE	→	AND ARE SKIPPED HERE
`claude -p "fix it"`	→	`claude --bare -p "fix it"`

3 A typo silently switches it off

Matchers are case-sensitive, and nothing warns you when nothing matches.

YOU WROTE	→	THE TOOL IS CALLED
`"matcher": "bash"`	→	`Bash`

4 Your hook is just a file on disk

The rule lives in a file, and Claude edits files. Deny those paths or it can rewrite its own leash.

YOUR POLICY LIVES IN	→	WHICH IS EDITABLE BY
`.claude/settings.json`	→	`Edit · Write`

A hook only stops what it can see, on the path where it is running.

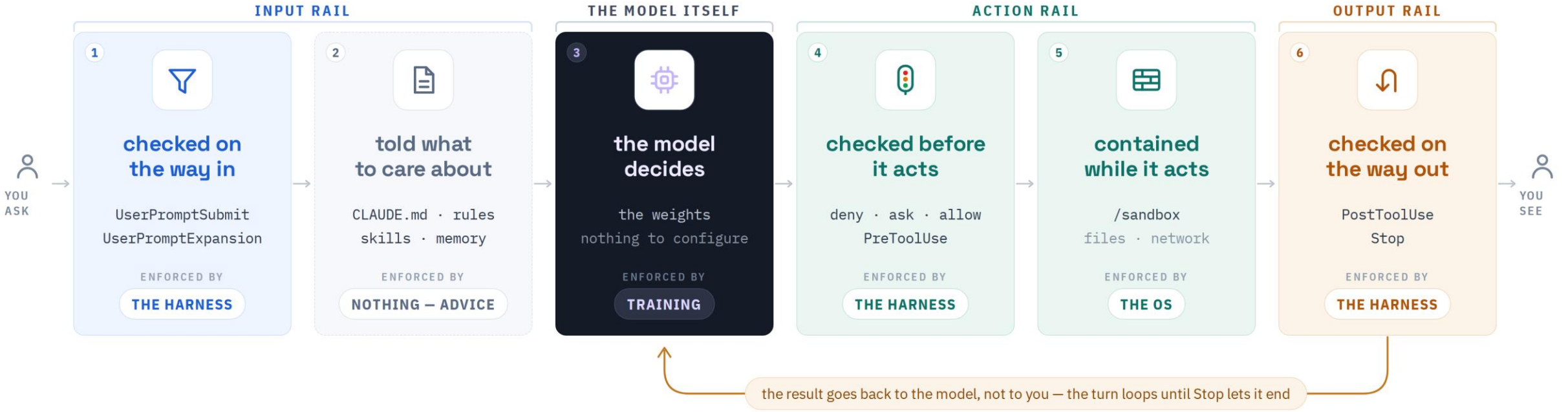
Nothing here is fixed by writing a better hook. If the rule has to hold no matter what, it belongs in permissions and the sandbox — below the level Claude can reach.

Building a Guardrail

Guardrails — checks that run around the model, not inside it

One turn, from your keyboard back to your screen: **three rails**, and **four places** to build them — prompt files **advise** · the harness **decides** · the OS **enforces** · the weights **judge**.

Events and settings per Claude Code docs
code.claude.com/docs · hooks · permissions · sandboxing



Guardrail · the policy

The boundary itself. What is allowed, what is forbidden, what must never happen — a company rule, written down.

E.G. **"Never expose customer data."**

ONE WAY TO ENFORCE IT →

permissions, the sandbox and CI enforce boundaries too — hooks are just the programmable one

Hook · the automation

Code you attach to a named event in the workflow. It does not decide policy; it carries one out, every time, without being asked.

E.G. `PostToolUse` → scan the output, strip anything sensitive.

Mitigating OWASP LLM Top10 using Hooks

Risk	Description	Hook Strategy
LLM01	**Prompt Injection**	PreToolUse guard: scan tool args before `Bash/Edit/Write` — catches indirect injection from fetched or RAG content
LLM02	**Sensitive Info Disclosure**	PreToolUse protect-secrets: block `reads/writes` of .env & keys; regex-scan tool_input for PII/secrets
LLM03	**Supply Chain**	Mostly upstream. Partial: PreToolUse if-filter to gate `Bash(npm install *) / pip install`
LLM04	**Data & Model Poisoning**	Not addressable at runtime — training-time / data-pipeline risk
LLM05	**Improper Output Handling**	PostToolUse (`Edit|Write`): run vuln/lint scanner; exit 2 to block or feed findings back to Claude
LLM06	**Excessive Agency**	PreToolUse block-dangerous-commands (`rm -rf, git push, curl | sh`); scope auto-approvals via PermissionRequest
LLM07	**System Prompt Leakage**	InstructionsLoaded audit of `CLAUDE.md` loads + protect-secrets — keep secrets out of prompt context
LLM08	**Vector / Embedding Weaknesses**	Only if agent does RAG via tools: PreToolUse on `mcp__.*__(search|query)` to validate inputs
LLM09	**Misinformation**	Not addressable at runtime. Partial: Stop / agent hook to verify tests pass or claims hold
LLM10	**Unbounded Consumption**	SubagentStart caps + rate limiting; per-hook timeout; token / turn budgets

Amber = not addressable by hooks at runtime (needs upstream model / data / retrieval controls)

Mitigating OWASP LLM Top10 using Hooks

Risk	Description	Hook Strategy
LLM01	**Prompt Injection**	PreToolUse guard: scan tool args before Bash/Edit/Write — catches indirect injection from fetched or RAG content
LLM02	**Sensitive Info Disclosure**	PreToolUse protect-secrets: block reads/writes of .env & keys; regex-scan tool_input for PII/secrets
LLM03	**Supply Chain**	Mostly upstream. Partial: PreToolUse if-filter to gate Bash(npm install *) / pip install *
LLM04	**Data & Model Poisoning**	Not addressable at runtime — training-time / data-pipeline risk
LLM05	**Improper Output Handling**	PostToolUse (Edit|Write): run vuln/lint scanner; exit 2 to block or feed findings back to Claude
LLM06	**Excessive Agency**	PreToolUse block-dangerous-commands (rm -rf, git push, curl | sh); scope auto-approvals via PermissionRequest
LLM07	**System Prompt Leakage**	InstructionsLoaded audit of CLAUDE.md loads + protect-secrets — keep secrets out of prompt context
LLM08	**Vector / Embedding Weaknesses**	Only if agent does RAG via tools: PreToolUse on mcp__.*__(search|query) to validate inputs
LLM09	**Misinformation**	Not addressable at runtime. Partial: Stop / agent hook to verify tests pass or claims hold
LLM10	**Unbounded Consumption**	SubagentStart caps + rate limiting; per-hook timeout; token / turn budgets

Q/A



Thank you!